

Generative Modeling from First Principles

ELBO, Adversarial Games, and Reverse Diffusion

Course materials

2026-07-06

Table of contents

1	Welcome	3
1.1	What this is	3
1.2	Prerequisites	3
1.3	Setup	3
1.4	How to use these materials	4
1.5	Tips for students	4
1.6	Structure	5
2	Generative Modeling from First Principles	6
2.0.1	ELBO, Adversarial Games, and Reverse Diffusion	6
2.0.2	Structure	6
2.1	Part 0 — One unifying framework	7
2.2	Part 1 — VAE	8
2.2.1	1.1 Why maximum likelihood becomes intractable	8
2.2.2	1.2 Deriving the ELBO	8
2.2.3	1.3 Making “KL regularization” <i>visible</i>	10
2.2.4	1.4 The <i>exact</i> reason L2 (Gaussian) decoders blur	12
2.2.5	1.5 Escaping blur: a menu of fixes (and VQ-VAE)	14
2.2.6	1.6 The conv upgrade: spatial latents beat flat MLPs	14
2.3	Part 2 — GAN	15
2.3.1	2.1 The minimax game and what it actually optimizes	15
2.4	Part 3 — Denoising score matching: predicting noise <i>is</i> learning the score	16
2.5	Part 4 — DDPM, derived	17
2.5.1	4.1 Forward process and its closed form	18
2.5.2	4.2 Why predict the noise? (and the $\epsilon / x_0 / v$ equivalence)	19
2.5.3	4.3 Reverse process: why a Gaussian, and its exact mean	19
2.5.4	4.4 Diagnostics instead of a cherry-picked strip	21
2.5.5	4.5 The modern diffusion family (why vanilla DDPM is only the start)	22
2.6	Part 5 — Comparing the objectives <i>quantitatively</i>	23
2.6.1	Reading the comparison (and its caveats)	24
2.7	Appendix — Common misconceptions (each debunked by a cell above)	24

1 Welcome

1.1 What this is

This is an interactive course on generative modeling built around three families of models:

- **VAEs** — variational inference, the ELBO, and why Gaussian decoders blur
- **GANs** — the minimax game, optimal discriminator, and measurable mode collapse
- **Diffusion** — score matching, DDPM forward/reverse derivations, and sampling

Every figure is tied to a **measurable claim**: we derive the math, implement it, and verify it numerically. No cherry-picked sample strips.

1.2 Prerequisites

- Python 3.10+ and basic NumPy / PyTorch familiarity
- Comfort with probability (expectations, KL divergence) at an introductory ML level
- A laptop CPU is enough — all models are small MLPs on 8×8 digits and finish in a few minutes

1.3 Setup

Create a virtual environment and install dependencies:

```
python -m venv .venv
source .venv/bin/activate # Windows: .venv\Scripts\activate
pip install -r requirements.txt
```

Install [Quarto](#) (one-time):

```
# macOS
brew install --cask quarto

# or download the installer from https://quarto.org/docs/download/
```

1.4 How to use these materials

Read online (HTML book with folded code and figures):

[Open the notebook chapter](#)

Download as PDF (~1.2 MB, all math and figures):

[Generative-Modeling-from-First-Principles.pdf](#)

Build locally:

```
quarto render
open _book/index.html
```

Live preview while editing:

```
quarto preview
```

Run the notebook interactively (re-executes all cells):

```
jupyter lab generative_first_principles.ipynb
```

Or re-render with fresh execution:

```
quarto render --execute
```

1.5 Tips for students

Variable	Default	Effect
QUICK	True in source build	Faster training; set False for nicer figures
SEED	0	Reproducible runs
IMG	8	Image side length (8×8 digits)

The notebook source is generated by `build_notebook.py`. After editing that file, run:

```
python build_notebook.py
quarto render
```

1.6 Structure

Part	Topic
0	One unifying framework — what object are we learning?
1	VAE — ELBO derivation, latent diagnostics, L2 blur, VQ-VAE
2	GAN — minimax, JS divergence, measured collapse
3	Denoising score matching
4	DDPM — forward/reverse derived, ancestral + DDIM sampling
5	Quantitative comparison + modern diffusion family
Appendix	Common misconceptions, each linked to a cell

Continue to the [full notebook](#) →

2 Generative Modeling from First Principles

2.0.1 ELBO, Adversarial Games, and Reverse Diffusion

This notebook is a rewrite of the usual “VAE, GAN, and Diffusion from scratch” tour with three rules:

1. **Every figure demonstrates a measurable claim.** No cherry-picked sample strips. If we assert “mode collapse”, we *measure* it; if we assert “the closed-form forward diffusion is correct”, we *verify* it numerically.
2. **Derive the math, then implement it.** The ELBO, the optimal GAN discriminator, the score/noise equivalence, and the DDPM forward/reverse equations are derived, not asserted.
3. **Stay beginner-accessible.** Intuition is kept, but it is always cashed out into an equation or an experiment.

Runtime. Everything runs on CPU in a few minutes. To keep it fast and dependency-light, all *image* models are small MLPs operating on 8×8 digits (from `torchvision` MNIST if available, otherwise `sklearn`’s `load_digits`, which requires no download). Swapping in full MNIST + conv nets is a localized change (IMG and the model classes).

2.0.2 Structure

- **Part 0** — One unifying framework: what object are we actually learning?
- **Part 1** — VAE: intractable MLE → ELBO derivation → latent-space diagnostics → the *exact* reason L2 decoders blur.
- **Part 2** — GAN: minimax → optimal discriminator → JS divergence → collapse & instability *measured*.
- **Part 3** — Denoising score matching: predicting noise → learning $\nabla_x \log p(x)$.
- **Part 4** — DDPM: forward/reverse derived from first principles, verified, sampled (ancestral + DDIM).
- **Part 5** — Comparing objectives quantitatively (class-entropy + a Fréchet feature distance) + the modern diffusion family.
- **Appendix** — Common misconceptions, each linked to the cell that debunks it.

device: mps | QUICK: False | torch: 2.9.1

data: torchvision MNIST (resized to 32x32) | shape: (60000, 1, 32, 32) | classes: 10

Sample of the working dataset



2.1 Part 0 — One unifying framework

A common opening line is “*all generative models map noise to data.*” That is true but lossy: it flattens distinctions that matter the moment you try to train or evaluate these models.

The object we actually want is a distribution over data,

$$p_{\theta}(x) \approx p_{\text{data}}(x),$$

and the three families differ in **what they commit to modeling**:

Model	What it learns	Likelihood $p_{\theta}(x)$?	Sampling
VAE	a latent-variable <i>likelihood</i> model $p_{\theta}(x z)p(z)$	tractable lower bound (ELBO)	ancestral: $z \sim p(z)$, $x \sim p_{\theta}(x z)$
GAN	an <i>implicit</i> sampler $G_{\theta}(z)$	undefined (no density)	one forward pass $x = G_{\theta}(z)$
DDPM	the transitions of a <i>reverse stochastic process</i>	tractable lower bound (also an ELBO!)	iterative denoising, T steps

Why the distinction matters in practice:

- The VAE decoder defines an explicit likelihood $p_\theta(x | z)$, so its failure modes are *likelihood* failure modes (e.g. a Gaussian likelihood forces the decoder toward a conditional **mean** — Part 1.4).
- A GAN defines **no** density. You cannot ask “what is $p_\theta(x)$?”; you can only sample. This is exactly why we must evaluate it with sample-based metrics (Part 2.2, Part 5).
- A DDPM commits to a *Markov chain*. Its training target and its samplers (ancestral vs DDIM, Part 4) are consequences of that chain, not of a single “noise→image” map.

Interestingly, VAE and DDPM both optimize an ELBO — a DDPM is, formally, a deep hierarchical VAE with a *fixed* (non-learned) encoder (the forward noising process). We’ll see the shared skeleton by Part 4.

2.2 Part 1 — VAE

2.2.1 1.1 Why maximum likelihood becomes intractable

We posit a latent-variable model: a prior $p(z) = \mathcal{N}(0, I)$ and a decoder $p_\theta(x | z)$. The marginal likelihood of a datapoint is

$$p_\theta(x) = \int p_\theta(x | z) p(z) dz.$$

To do maximum likelihood we would maximize $\sum_x \log p_\theta(x)$, but the integral is intractable:

- z is continuous and high-dimensional, so there is no closed form for a neural $p_\theta(x | z)$.
- A naive Monte Carlo estimate $p_\theta(x) \approx \frac{1}{S} \sum_s p_\theta(x | z^{(s)})$, $z^{(s)} \sim p(z)$ has catastrophic variance: for almost every prior sample $z^{(s)}$, $p_\theta(x | z^{(s)}) \approx 0$. The posterior $p_\theta(z | x)$ is concentrated on a tiny region of latent space that prior samples essentially never hit.

So we need to (a) avoid the integral and (b) sample z from something close to the *posterior*. Both are solved by variational inference.

2.2.2 1.2 Deriving the ELBO

Introduce an **approximate posterior** $q_\phi(z | x)$ (the “encoder”). For *any* such q_ϕ we can decompose the log-likelihood exactly. Start from the definition and multiply/divide by q_ϕ :

$$\log p_\theta(x) = \log \int p_\theta(x, z) dz = \log \int q_\phi(z | x) \frac{p_\theta(x, z)}{q_\phi(z | x)} dz = \log \mathbb{E}_{q_\phi} \left[\frac{p_\theta(x, z)}{q_\phi(z | x)} \right].$$

Exact identity (no inequality yet). Because $\log p_\theta(x)$ does not depend on z ,

$$\log p_\theta(x) = \mathbb{E}_{q_\phi}[\log p_\theta(x)] = \mathbb{E}_{q_\phi}\left[\log \frac{p_\theta(x, z)}{q_\phi(z | x)}\right] + \mathbb{E}_{q_\phi}\left[\log \frac{q_\phi(z | x)}{p_\theta(z | x)}\right],$$

where we used $p_\theta(x, z) = p_\theta(z | x)p_\theta(x)$. The two terms are

$$\boxed{\log p_\theta(x) = \underbrace{\mathbb{E}_{q_\phi}\left[\log \frac{p_\theta(x, z)}{q_\phi(z | x)}\right]}_{\mathcal{L}(x) \text{ (ELBO)}} + \underbrace{D_{\text{KL}}(q_\phi(z | x) \| p_\theta(z | x))}_{\geq 0}}$$

Since $\text{KL} \geq 0$, $\mathcal{L}(x) \leq \log p_\theta(x)$ — hence **Evidence Lower Bound**. The gap is exactly how wrong our approximate posterior is. Maximizing $\mathcal{L}$ simultaneously (i) pushes up a lower bound on the data likelihood and (ii) pulls q_ϕ toward the true posterior.

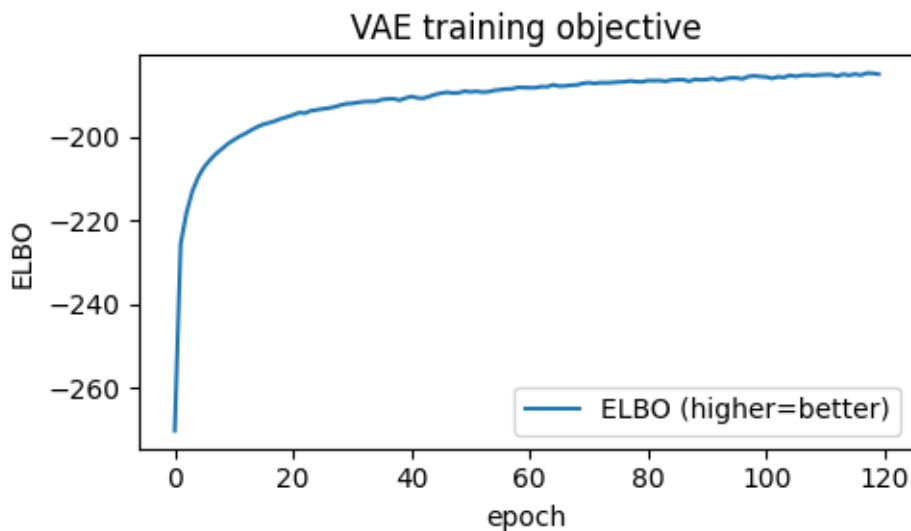
Rewriting into the trainable form. Split $p_\theta(x, z) = p_\theta(x | z)p(z)$:

$$\mathcal{L}(x) = \mathbb{E}_{q_\phi}[\log p_\theta(x | z)] - \underbrace{\mathbb{E}_{q_\phi}\left[\log \frac{q_\phi(z | x)}{p(z)}\right]}_{D_{\text{KL}}(q_\phi(z|x) \| p(z))} = \boxed{\mathbb{E}_{q_\phi}[\log p_\theta(x | z)] - D_{\text{KL}}(q_\phi(z | x) \| p(z))}$$

- **Term 1** (reconstruction): decode $z \sim q_\phi(z | x)$ and score the data. This is where the *choice of likelihood* bites (Part 1.4).
- **Term 2** (rate / regularizer): keep the per-example posterior near the prior. With $q_\phi = \mathcal{N}(\mu, \text{diag}(\sigma^2))$ and $p = \mathcal{N}(0, I)$ this KL is closed-form: $\frac{1}{2} \sum_j (\mu_j^2 + \sigma_j^2 - \log \sigma_j^2 - 1)$.

Reparameterization. Term 1 is an expectation over q_ϕ , whose parameters we differentiate. Writing $z = \mu_\phi(x) + \sigma_\phi(x) \odot \epsilon$, $\epsilon \sim \mathcal{N}(0, I)$ moves the randomness off the parameters so ∇_ϕ can pass through the sample (a low-variance pathwise gradient instead of a high-variance score-function estimator).

epoch	24		-ELBO	193.90		recon	187.29		KL	6.614
epoch	48		-ELBO	189.65		recon	182.76		KL	6.898
epoch	72		-ELBO	187.37		recon	180.32		KL	7.052
epoch	96		-ELBO	186.06		recon	178.90		KL	7.159
epoch	120		-ELBO	185.16		recon	177.93		KL	7.233



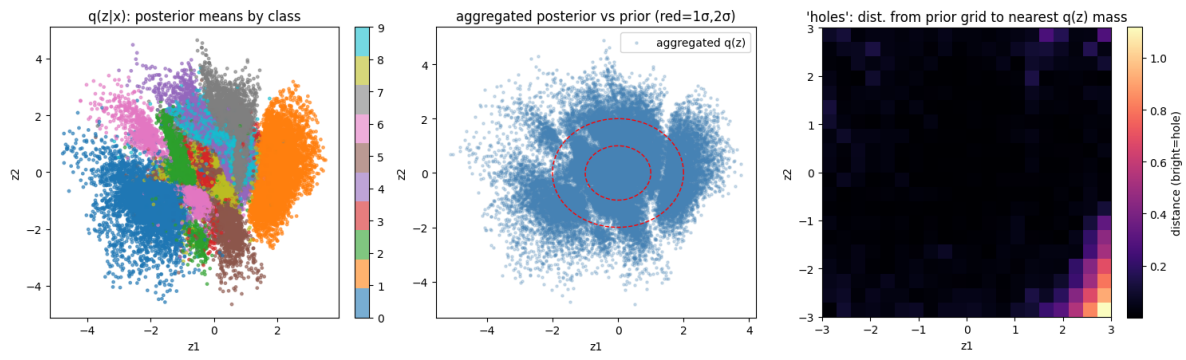
VAE reconstructions (row 1: input, row 2: $E[x|z]$)



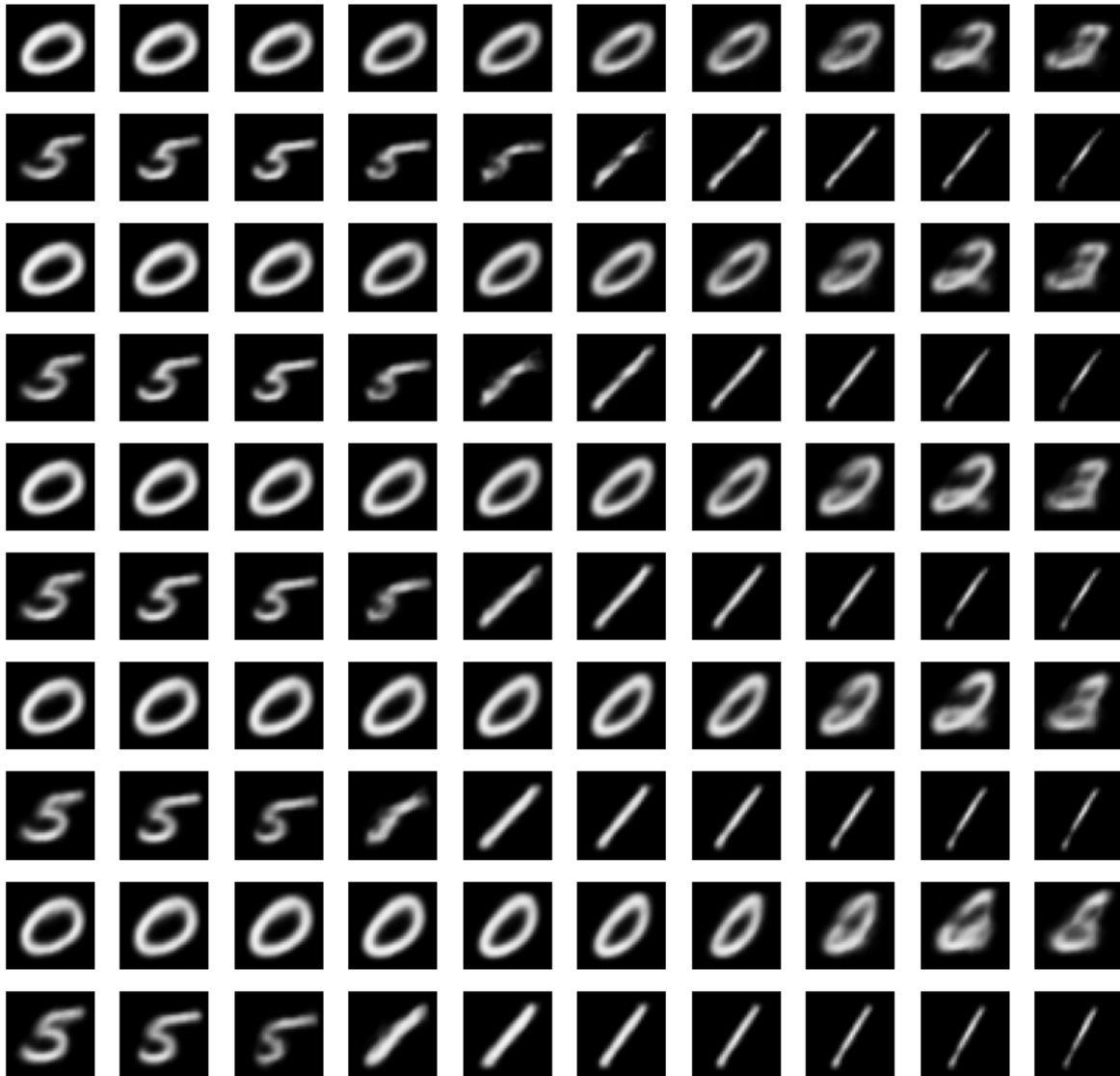
2.2.3 1.3 Making “KL regularization” *visible*

Textbooks say the KL term “keeps latent space nice” and warns about “holes” (regions of the prior that the model never mapped real data to). With a 2D latent we can *see* both. We measure three things:

1. **Per-example posteriors** $q_\phi(z | x)$ — a scatter of $\mu_\phi(x)$ colored by class.
2. **Aggregated posterior** $q_\phi(z) = \frac{1}{N} \sum_x q_\phi(z | x)$ vs the prior $\mathcal{N}(0, I)$. The ELBO only regularizes each posterior *individually*; there is no guarantee the *aggregate* matches the prior. Mismatch — sampling from $p(z)$ lands in low-density regions.
3. **Holes**: decode a grid over latent space and overlay where the aggregated posterior actually has mass. Grid points far from that mass are “holes” — the prior samples them, but the model never trained there, so decodes are garbage/blurry.



decoding a 10x10 latent grid — bright 'hole' regions decode to mush



Fraction of prior grid points >0.5 from any real latent (holes): 0.02

2.2.4 1.4 The *exact* reason L2 (Gaussian) decoders blur

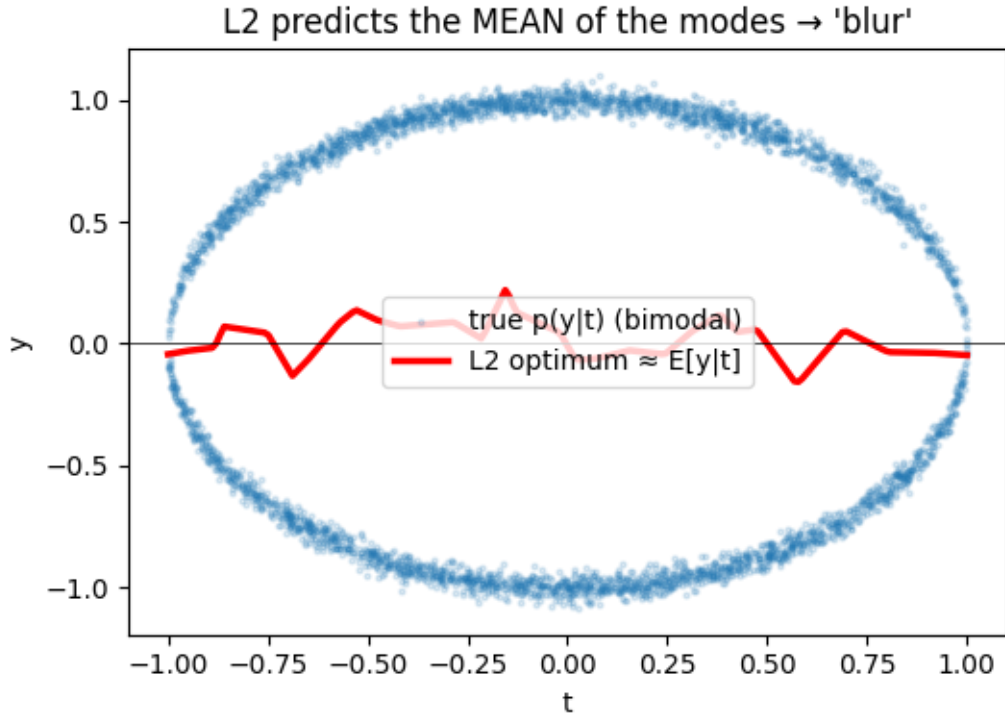
The usual claim “L2 loss causes blur” is a symptom, not a mechanism. Here is the mechanism, exactly.

If the decoder uses a Gaussian likelihood with fixed variance, $p_\theta(x | z) = \mathcal{N}(x; f_\theta(z), \sigma^2 I)$, then maximizing $\log p_\theta(x | z)$ is minimizing $\|x - f_\theta(z)\|^2$ — the L2 objective. The population minimizer of a squared error is a **conditional expectation**:

$$f^*(z) = \arg \min_f \mathbb{E}[\|x - f(z)\|^2] = \mathbb{E}[x | z].$$

Now suppose a single code z is consistent with **multiple** distinct plausible images (the true $p(x | z)$ is multimodal — e.g. a “4” that could be open- or closed-top). The optimal L2 decoder cannot output “either/or”; it must output the **average of the modes**, $\mathbb{E}[x | z]$, which is a blurry blend of both. Blur is not a numerical artifact — it is the *Bayes-optimal* output of a mean predictor under multimodal uncertainty.

We demonstrate this with a task where we know the ground truth is bimodal, so the blame is unambiguous.



mean |prediction| (dist to mode-average line y=0): 0.061

mean dist from prediction to a real mode: 0.765

The fit hugs the between-modes average, not either mode → that is the blur, exactly.

2.2.5 1.5 Escaping blur: a menu of fixes (and VQ-VAE)

Section 1.4 pinned blur on the **Gaussian-mean decoder**: when $p(x | z)$ is multimodal, the Bayes-optimal squared-error predictor outputs $\mathbb{E}[x | z]$ — a blend. That is the principled mechanism. (Our digit VAE already uses a Bernoulli pixel likelihood, which avoids the L2 Gaussian equivalence, but small MLPs on blurry 32×32 inputs can still look soft for capacity and posterior-hole reasons.)

Common escape routes:

Fix	Idea	Trade-off
Different likelihood	Bernoulli / discretized logistic instead of Gaussian L2	Still averages if the decoder is effectively unimodal per z
GAN (Part 2)	No explicit $p_\theta(x)$; discriminator rewards sharp, realistic samples	No density; mode collapse; we'll study this next
β-VAE / less KL	Weaken the $\mathcal{N}(0, I)$ prior pressure	Helps “holes” and oversmoothing from KL, not the multimodal-mean issue by itself
VQ-VAE	Discrete latents: encoder snaps z to nearest codebook vector	No continuous Gaussian KL; decoder learns one map per code — <i>implemented below</i>
Diffusion (Part 4)	Score matching at many noise levels	Sharp samples possible, but many sampling steps

VQ-VAE (van den Oord et al.) replaces $z \in \mathbb{R}^d$ with a **finite codebook** $\{e_k\}_{k=1}^K$. The encoder outputs z_e ; we set $z_q = e_{k^*}$ where $k^* = \arg \min_k \|z_e - e_k\|$. Training uses a straight-through estimator on z_q and a commitment loss — **no KL on a continuous latent**, so the standard “Gaussian posterior $\rightarrow$ blur” story does not apply in the same way.

Caveat: VQ-VAE does **not** ship with a simple generative prior like $\mathcal{N}(0, I)$. The original paper learns a separate autoregressive model over codes. Picking **uniform random** codebook indices is a common demo mistake — most entries are never used during training and decode to mush. Below we (i) compare **reconstructions** on the same inputs (the fair sharpness check), and (ii) sample codes from the **empirical marginal** $p(k)$ on training data.

We will look at **GANs** in Part 2.

2.2.6 1.6 The conv upgrade: spatial latents beat flat MLPs

The linear VQ-VAE above proves the **discrete-latent** idea: once the codebook is initialized at the right scale and actually used, blur from continuous Gaussian averaging largely disappears.

The remaining soft edges are mostly a **topology** issue — `nn.Linear` on flattened pixels has no notion of locality.

A **convolutional VQ-VAE** keeps the same `VectorQuantizer`, but the encoder produces a **spatial** latent map (B, C, H, W) . Each $H \times W$ location snaps to its own codebook entry, so the decoder can mix local stroke fragments instead of memorizing whole images in one vector. This is the architecture used in the original VQ-VAE paper and in systems like VQ-GAN.

Below we train a minimal `Conv2d` / `ConvTranspose2d` variant on the same data and compare reconstructions to the flat MLP VQ-VAE.

2.3 Part 2 — GAN

2.3.1 2.1 The minimax game and what it actually optimizes

A GAN pairs a generator G (implicit sampler, $x = G(z)$, $z \sim p_z$) with a discriminator $D(x) \in (0, 1)$ estimating “probability x is real”. The value function is

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{\text{data}}} [\log D(x)] + \mathbb{E}_{z \sim p_z} [\log(1 - D(G(z)))].$$

Optimal discriminator (for fixed G). Let p_g be the generator’s distribution. Then

$$V = \int p_{\text{data}}(x) \log D(x) dx + \int p_g(x) \log(1 - D(x)) dx.$$

Pointwise, maximize $a \log D + b \log(1 - D)$ over D : derivative $\frac{a}{D} - \frac{b}{1-D} = 0 \Rightarrow D = \frac{a}{a+b}$. Hence

$$D^*(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_g(x)}.$$

What G then minimizes. Substituting D^* and adding/subtracting $\log 2$:

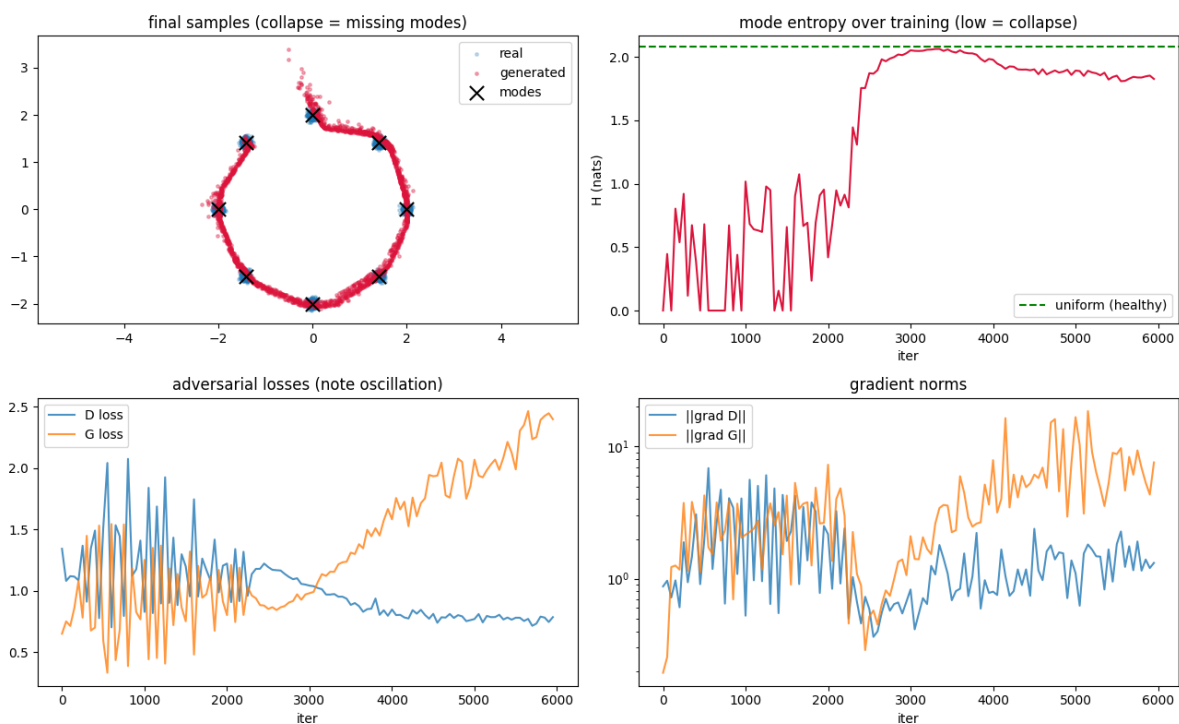
$$V(D^*, G) = -\log 4 + \text{KL}\left(p_{\text{data}} \left\| \frac{p_{\text{data}} + p_g}{2}\right.\right) + \text{KL}\left(p_g \left\| \frac{p_{\text{data}} + p_g}{2}\right.\right) = -\log 4 + 2 \text{JS}(p_{\text{data}} \| p_g).$$

So at the discriminator’s optimum the generator minimizes the **Jensen–Shannon divergence**, with global optimum $p_g = p_{\text{data}}$ (value $-\log 4$). This is far more informative than “forger vs detective”: it tells us the *loss geometry*, and it hints at the pathologies — when p_g and p_{data} have disjoint support, JS is constant ($\log 2$) and gradients vanish; and nothing in the objective

forbids p_g from dropping modes (**mode collapse**), because a G that nails a subset of modes can still fool a lagging D .

max possible entropy (uniform over 8 modes): 2.0794415416798357

final: coverage 8/8 modes | entropy 1.827 / 2.079



per-mode counts (of 3000): [216 276 171 1025 191 192 161 163]

fraction of samples on any mode: 0.80

Collapse is now a NUMBER (entropy, coverage), not a vibe from a sample strip.

2.4 Part 3 — Denoising score matching: predicting noise *is* learning the score

Before DDPM, the cleaner idea: learn the **score** $s(x) = \nabla_x \log p(x)$ — the vector field pointing toward higher data density. With the score you can sample by **Langevin dynamics**:

$$x_{k+1} = x_k + \frac{\eta}{2} \nabla_x \log p(x_k) + \sqrt{\eta} \xi_k, \quad \xi_k \sim \mathcal{N}(0, I),$$

which converges to $p(x)$ as $\eta \rightarrow 0, k \rightarrow \infty$.

We can't evaluate $\nabla_x \log p$ (we don't know p), but **denoising score matching** (Vincent, 2011) gives a tractable target. Corrupt data with Gaussian noise, $\tilde{x} = x + \sigma\epsilon$, $\epsilon \sim \mathcal{N}(0, I)$, so $q_\sigma(\tilde{x} | x) = \mathcal{N}(\tilde{x}; x, \sigma^2 I)$. A key identity: the minimizer of the denoising objective equals the score of the *noised* density,

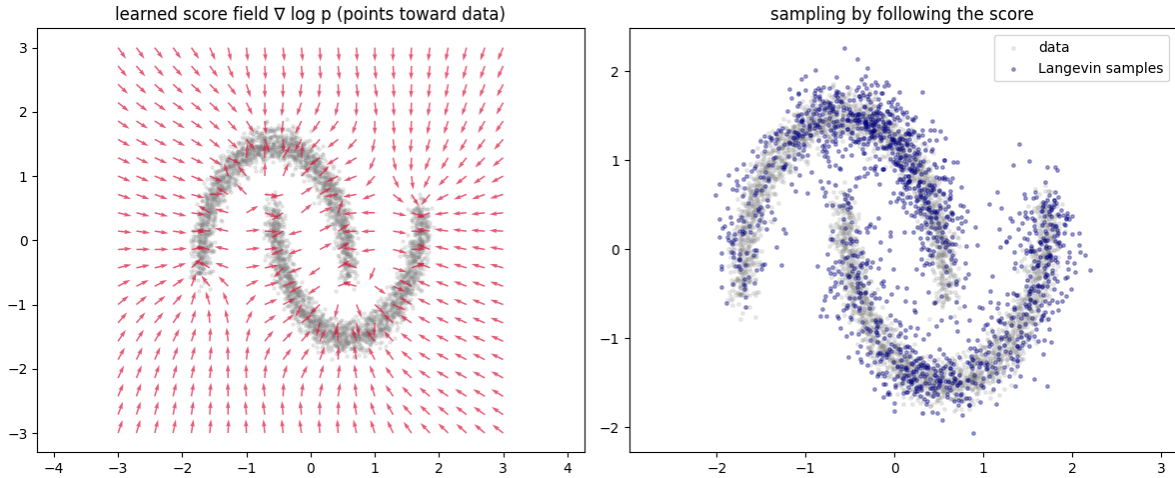
$$s_\theta^*(\tilde{x}) = \nabla_{\tilde{x}} \log q_\sigma(\tilde{x}) = \mathbb{E} \left[\frac{x - \tilde{x}}{\sigma^2} \middle| \tilde{x} \right].$$

Because $\nabla_{\tilde{x}} \log q_\sigma(\tilde{x} | x) = (x - \tilde{x})/\sigma^2 = -\epsilon/\sigma$, training a network to predict the score is *equivalent* to training it to predict the noise ϵ (up to the $-1/\sigma$ factor):

$$\underbrace{\mathbb{E} \left\| s_\theta(\tilde{x}) - \frac{x - \tilde{x}}{\sigma^2} \right\|^2}_{\text{score matching}} \iff \underbrace{\mathbb{E} \left\| \epsilon_\theta(\tilde{x}) - \epsilon \right\|^2}_{\text{noise prediction, with } s_\theta = -\epsilon_\theta/\sigma}.$$

This is the bridge to diffusion: the DDPM training loss in Part 4 is exactly noise prediction, i.e. score matching at many noise levels at once.

final DSM loss: 29.1240291595459



mean dist gen->data: 0.045 | data->data nn: 0.017

2.5 Part 4 — DDPM, derived

A DDPM defines a **fixed** forward chain that gradually turns data into noise, then **learns** the reverse chain.

2.5.1 4.1 Forward process and its closed form

Forward (variance-preserving) Markov chain with schedule $\beta_1, \dots, \beta_T$:

$$q(x_t | x_{t-1}) = \mathcal{N}(x_t; \sqrt{1 - \beta_t} x_{t-1}, \beta_t I).$$

Let $\alpha_t = 1 - \beta_t$ and $\bar{\alpha}_t = \prod_{s \leq t} \alpha_s$. **Claim (closed form):**

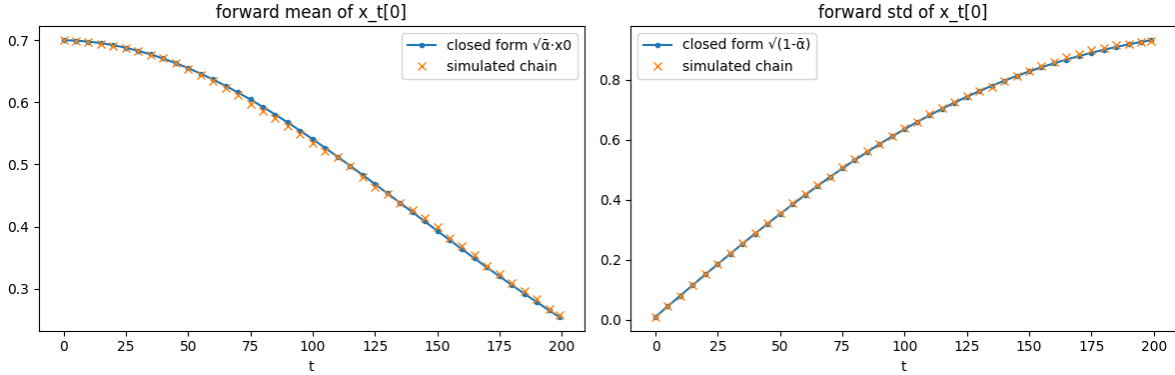
$$q(x_t | x_0) = \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t} x_0, (1 - \bar{\alpha}_t)I), \quad x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, \quad \epsilon \sim \mathcal{N}(0, I).$$

Derivation (induction). One step: $x_t = \sqrt{\alpha_t} x_{t-1} + \sqrt{1 - \alpha_t} \epsilon_t$. Assume $x_{t-1} = \sqrt{\bar{\alpha}_{t-1}} x_0 + \sqrt{1 - \bar{\alpha}_{t-1}} \bar{\epsilon}_{t-1}$. Substitute:

$$x_t = \sqrt{\alpha_t \bar{\alpha}_{t-1}} x_0 + \underbrace{\sqrt{\alpha_t (1 - \bar{\alpha}_{t-1})} \bar{\epsilon}_{t-1} + \sqrt{1 - \alpha_t} \epsilon_t}_{\text{sum of independent Gaussians}}.$$

The noise term is zero-mean Gaussian with variance $\alpha_t(1 - \bar{\alpha}_{t-1}) + (1 - \alpha_t) = 1 - \alpha_t \bar{\alpha}_{t-1} = 1 - \bar{\alpha}_t$. And $\alpha_t \bar{\alpha}_{t-1} = \bar{\alpha}_t$. So $x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon$. ■

Below we **verify this numerically**: iterating the one-step kernel many times must match the closed form's mean and variance.



max |mean error|: 0.006610393524169922

max |std error|: 0.008877594753495033

2.5.2 4.2 Why predict the noise? (and the ϵ / x_0 / v equivalence)

Given $x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon$, the three common prediction targets are **deterministically interchangeable** given (x_t, t) :

$$x_0 = \frac{x_t - \sqrt{1 - \bar{\alpha}_t}\epsilon}{\sqrt{\bar{\alpha}_t}}, \quad \epsilon = \frac{x_t - \sqrt{\bar{\alpha}_t}x_0}{\sqrt{1 - \bar{\alpha}_t}}, \quad v \equiv \sqrt{\bar{\alpha}_t}\epsilon - \sqrt{1 - \bar{\alpha}_t}x_0.$$

- **ϵ -prediction** (Ho et al. 2020) is the default: the target has unit variance at every t , so the loss is well-scaled across noise levels — a big practical reason it beats predicting x_0 directly.
- **x_0 -prediction** is intuitive but poorly scaled at high noise (target barely constrained).
- **v -prediction** (Salimans & Ho 2022) behaves well across the whole schedule (especially near $t=T$ and for distillation) and is what many modern systems use.

These are *reparameterizations of the same objective*, not different models. We verify the identities numerically.

```

||x0 - x0(eps)|| : 2.384185791015625e-07
||eps - eps(x0)||: 4.76837158203125e-07
||x0 - x0(v)||   : 4.76837158203125e-07
||eps - eps(v)|| : 4.76837158203125e-07
All ~0 => the three targets carry identical information given (x_t, t).

```

2.5.3 4.3 Reverse process: why a Gaussian, and its exact mean

We want $p_\theta(x_{t-1} | x_t)$. Two facts make a Gaussian reverse principled:

1. When each β_t is small, the true reverse conditional $q(x_{t-1} | x_t)$ is *approximately Gaussian* (small steps locally Gaussian), so $p_\theta(x_{t-1} | x_t) = \mathcal{N}(\mu_\theta, \Sigma_\theta)$ is a good family.
2. The *forward posterior conditioned on x_0* , $q(x_{t-1} | x_t, x_0)$, is **exactly** Gaussian and gives us the training target. By Bayes with the Gaussians above,

$$q(x_{t-1} | x_t, x_0) = \mathcal{N}(x_{t-1}; \tilde{\mu}_t(x_t, x_0), \tilde{\beta}_t I), \quad \tilde{\beta}_t = \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t} \beta_t,$$

$$\tilde{\mu}_t(x_t, x_0) = \frac{\sqrt{\bar{\alpha}_{t-1}} \beta_t}{1 - \bar{\alpha}_t} x_0 + \frac{\sqrt{\bar{\alpha}_t} (1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t} x_t.$$

Substituting $x_0 = (x_t - \sqrt{1 - \bar{\alpha}_t}\epsilon)/\sqrt{\bar{\alpha}_t}$ and simplifying gives the sampler's mean in terms of a **noise predictor** ϵ_θ :

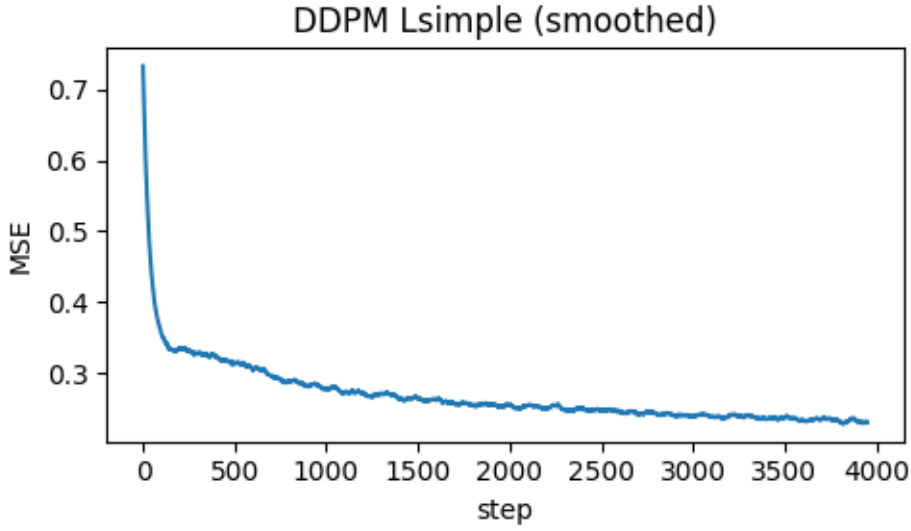
$$\mu_{\theta}(x_t, t) = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_{\theta}(x_t, t) \right).$$

Minimizing the ELBO for this model reduces (dropping t -dependent weights, as Ho et al. do) to the beautifully simple

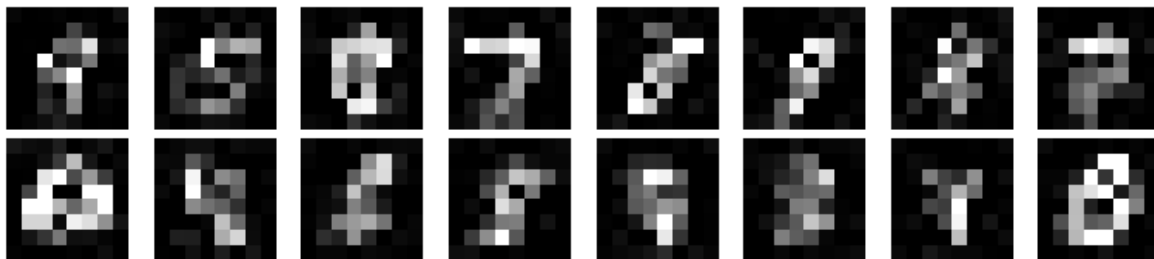
$$L_{\text{simple}} = \mathbb{E}_{x_0, t, \epsilon} \left\| \epsilon - \epsilon_{\theta}(\sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, t) \right\|^2.$$

That is exactly the score-matching / noise-prediction loss from Part 3, applied at all noise levels. We now train it.

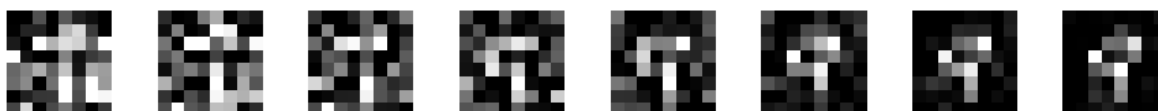
```
step   800 | Lsimple 0.2988
step  1600 | Lsimple 0.2633
step  2400 | Lsimple 0.2493
step  3200 | Lsimple 0.2386
step  4000 | Lsimple 0.2313
```



DDPM ancestral samples (16 independent seeds)



one reverse trajectory: noise -> image (left to right)



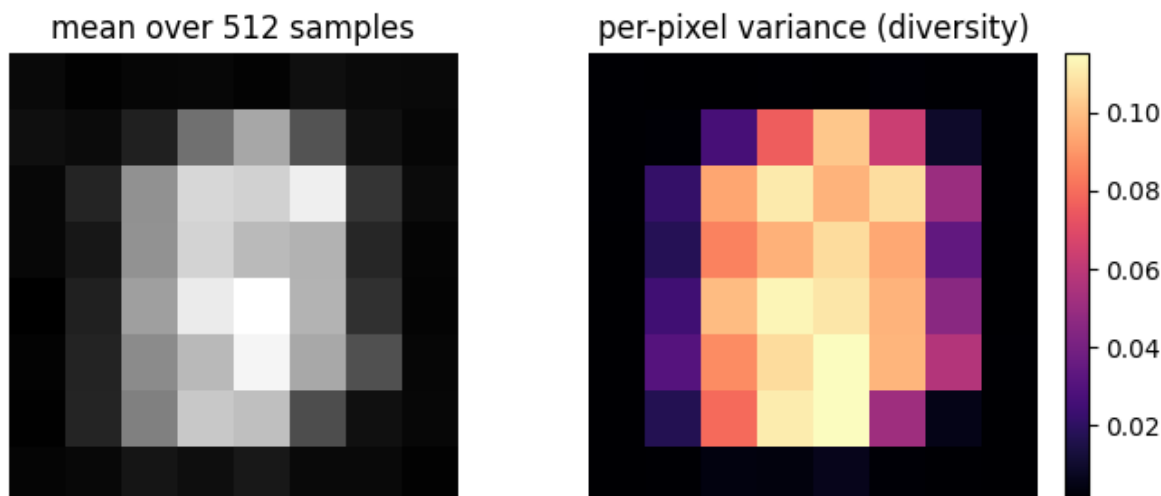
DDIM samples (25 steps, deterministic) — ~8x fewer steps



2.5.4 4.4 Diagnostics instead of a cherry-picked strip

A single sample strip tells you nothing about coverage or determinism. Three measurable diagnostics:

- **Per-pixel sample variance** across many seeds: a healthy model produces *diverse* samples (nonzero, structured variance), not the same image.
- **Ancestral vs DDIM agreement:** DDIM ($\eta = 0$) is deterministic given the seed; with the *same* initial noise it should track the ancestral mean. We quantify the step-count/quality trade-off with a metric.
- **A Fréchet feature distance over training** (defined in Part 5): sample quality should *improve* as training proceeds. We compute it at several checkpoints.



total sample variance (sum of per-pixel var): 2.578 (0 would mean collapse)

2.5.5 4.5 The modern diffusion family (why vanilla DDPM is only the start)

Vanilla ancestral DDPM needs hundreds–thousands of network calls. The field moved on; the essentials:

- **DDIM** (implemented above): reinterprets the same trained ϵ_θ as a *deterministic* ODE-like sampler on a subsequence of steps $\rightarrow$ 10–50 steps with little quality loss. Also gives a meaningful latent space (fixed noise $\rightarrow$ fixed image).
- **Latent diffusion (Stable Diffusion)**: run diffusion in a pretrained autoencoder’s *latent* space, not pixels. Cheaper and higher-res; diffusion models the compressed code, a decoder renders pixels.
- **Flow matching / rectified flow**: instead of a noising Markov chain, learn a velocity field $v_\theta(x, t)$ transporting noise to data along (nearly) straight paths, trained by regressing to a simple conditional target. Straighter paths $\rightarrow$ fewer sampling steps; this underlies several current SOTA systems.
- **Consistency models**: distill/train a network that maps *any* point on a trajectory directly to its endpoint, enabling **1–4 step** generation.

The through-line: they all learn something equivalent to the score / a probability-flow transport, then differ in the *path* and the *sampler*. The ϵ -prediction loss you trained above is the common ancestor.

2.6 Part 5 — Comparing the objectives *quantitatively*

We now put VAE, GAN, and DDPM samples on the same footing with two sample-based metrics (the only fair option, since a GAN has no likelihood):

1. **Class entropy** — feed generated images to a classifier trained on real data, histogram predicted classes, compute $H = -\sum_i p_i \log p_i$. High = balanced coverage; low = mode collapse.
2. **Fréchet feature distance (FD)** — an FID-analogue. Using the classifier’s penultimate features, fit Gaussians to real and generated features and compute

$$\text{FD} = \|\mu_r - \mu_g\|^2 + \text{Tr}(\Sigma_r + \Sigma_g - 2(\Sigma_r \Sigma_g)^{1/2}).$$

Lower = generated feature statistics match real. (We use our own small classifier, not InceptionV3, so absolute numbers aren’t comparable to published FID — but *relative* comparisons here are meaningful.)

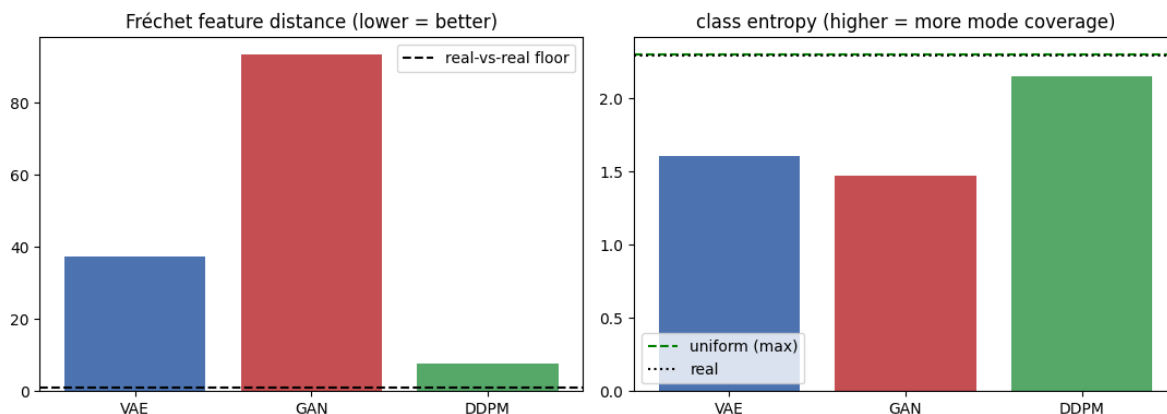
First, train the classifier and define the metrics.

```
classifier train accuracy: 0.946
max entropy (uniform): 2.302585092994046
```

image GAN samples



model	FD (lower better)	class-entropy	
REAL	0.912	2.292	<- reference
VAE	37.443	1.604	
GAN	93.397	1.473	
DDPM	7.672	2.153	



2.6.1 Reading the comparison (and its caveats)

- Interpret **relative**, not absolute, numbers: the FD floor is the real-vs-real value (a fresh real sample vs another real sample); no model can beat it.
- On these tiny MLPs + 8×8 data, expect: **VAE** — decent FD, tends to *lower* entropy only if the aggregated-posterior holes hurt sampling; samples look smooth/blurry (Part 1.4). **GAN** — can have sharp samples but is the one prone to *low entropy* (mode collapse, Part 2). **DDPM** — usually the best coverage (entropy near real) at the cost of many sampling steps.
- Results vary with seed and the QUICK budget — that variance is itself the point of Part 2’s instability discussion. Rerun with `QUICK=False` for cleaner separation.

The headline: because a GAN has no likelihood, *sample-based* metrics are the only common yardstick — and they turn vague claims (“sharper”, “more diverse”, “collapsed”) into numbers.

2.7 Appendix — Common misconceptions (each debunked by a cell above)

1. **“All generative models are just noise→image maps.”** They commit to different objects: a likelihood model (VAE), an implicit sampler with *no* density (GAN), and a reverse stochastic process (DDPM). — *Part 0*.
2. **“The ELBO is a heuristic.”** It is an *exact* decomposition $\log p_\theta(x) = \mathcal{L}(x) + D_{\text{KL}}(q_\phi \| p_\theta(z | x))$; the “bound” is just $\text{KL} = 0$. — *Part 1.2*.
3. **“KL just keeps latent space tidy.”** It regularizes each posterior *individually*; the **aggregated** posterior can still mismatch the prior, creating **holes** you sample from at generation time. — *Part 1.3*.

4. **“L2 causes blur (somehow).”** A Gaussian-mean decoder is Bayes-optimal at $\mathbb{E}[x | z]$; under multimodal uncertainty that is literally the average of the modes. Demonstrated on a bimodal target where the fit provably sits between the modes. — *Part 1.4.*
5. **“GAN = forger vs detective, the end.”** At D^* the generator minimizes $2\text{JS}(p_{\text{data}} \| p_g) - \log 4$; disjoint supports vanishing gradients, and nothing penalizes dropped modes. — *Part 2.1.*
6. **“You can eyeball mode collapse.”** Better to *measure* it: mode-assignment entropy and coverage over training. — *Part 2.2.*
7. **“Predicting noise is a trick.”** It equals learning the score $\nabla_x \log p$ of the noised data (up to $-1/\sigma$). — *Part 3.*
8. **“The DDPM closed form is just stated.”** Derived by induction and **verified** by matching a simulated chain’s mean/variance. — *Part 4.1.*
9. **“Predicting ϵ vs x_0 vs v are different models.”** They are interchangeable reparameterizations given (x_t, t) (verified to numerical precision); the difference is loss *scaling*/stability. — *Part 4.2.*
10. **“DDPM is the state of the art.”** It’s the foundation; DDIM, latent diffusion, flow matching / rectified flow, and consistency models dominate now. — *Part 4.5.*
11. **“Just look at the samples.”** Since a GAN has no likelihood, compare families with sample-based metrics (class entropy, Fréchet feature distance). — *Part 5.*

Rerun with `QUICK=False` for higher-fidelity training. Swap `IMG` and the model classes to move from 8×8 MLPs to full MNIST + conv nets.